

APZ lab4 report

High-Level Architecture Shift

Previous Architecture: A strictly Synchronous HTTP architecture with static routing. The facade-service communicated directly with the logging-service and counter-service via blocking HTTP requests. If any service went down, the request would fail or hang. Hazelcast was only used as a distributed map for logs.

New Architecture: a hybrid asynchronous architecture with Dynamic Service Discovery. The facade-service now resolves IPs dynamically via a central registry. Communication with the counter-service has been decoupled using an Asynchronous Message Queue (Hazelcast Queue), ensuring that if the database or counter-service goes offline, messages are safely buffered and processed later.

File content explanation in comparison to the previous lab

1. config-server/main.py

IP addresses and URLs for services were hardcoded into the docker-compose.yml and .env variables. This file acts as a Service Registry. It is a lightweight FastAPI application that holds an in-memory dictionary mapping service names to their active IP addresses.

2. facade-service/main.py

Previously: Maintained a static, comma-separated list of logging-service URLs loaded from environment variables and executed a synchronous HTTP POST to the counter-service. If the counter-service was slow or dead, the facade-service would either block or throw an HTTP 500/503 error to the client.

Now: Dynamic Routing. Before making any requests, it queries the config-server to get the latest, active list of URLs for both the logging and counter services.

Message Queue Producer: The synchronous HTTP POST to the counter-service is completely removed. Instead, it connects to the Hazelcast cluster and drops the payload into a distributed messages_queue. The response is immediately returned to the client as "ok", vastly improving response times and fault tolerance.

3. counter-service/main.py

Previously: Acted as a standard REST API. It exposed a POST /message endpoint that synchronously opened a PostgreSQL connection, inserted the message, committed the transaction, and returned a response.

Now: Message Queue Consumer. The POST /message endpoint is gone. Instead on startup it connects to Hazelcast and spawns a background Daemon Thread. This thread continuously listens to the messages_queue. When a message arrives, it pulls it off the queue and writes it to PostgreSQL asynchronously.

Self-Registration: On startup it runs an asynchronous loop to constantly attempt to register its ADVERTISED_URL with the config-server so the facade knows where to send GET requests.

4. logging-service/main.py

Previously: A simple service that received HTTP POSTs, saved them to a Hazelcast Distributed Map, and returned them on a GET request. It was entirely oblivious to the rest of the network.

Now: The core logging logic remains the same (still using the Hazelcast Map), but it now includes a Self-Registration mechanism. When the container starts, an asynchronous starts and pings the config-server with its SERVICE_NAME and ADVERTISED_URL so the facade can discover it dynamically for load balancing.

5. docker-compose.yml

Previously: Defined the static LOGGING_NODES and MESSAGES_URL for the facade-service. The architecture was rigid; adding a fourth logging service required changing the facade's configuration and restarting it.

Now: Added the new config-server container. Injected new environment variables (CONFIG_SERVER, SERVICE_NAME, ADVERTISED_URL) into every microservice.

The architecture is now highly scalable. If you add logging-service-4 to the compose file, it will automatically register itself with the config-server, and the facade-service will begin sending traffic to it without any restarts or manual configuration updates.

Executed a bash loop sending 10 POST requests (msg1–msg10) to `facade-service` on port 8000. Each response returns `{"status": "ok", "uuid": "..."} immediately proving the facade does not block on counter-service writes and acts purely as a message queue producer`

```
• (.venv APZ general) julfy@julyf-Nitro-AN515-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$ for i in {1..10}; do curl -X POST -H "Content-Type: application/json" -d '{"msg": "\msg$i"}' http://localhost:8000/facade; echo ""; done
{"status": "ok", "uuid": "e6f5b6ef-452e-44e3-bdc4-34bd1e8a6f04"}
{"status": "ok", "uuid": "33056a56-28f4-408f-a566-e0011b94feda"}
{"status": "ok", "uuid": "62aff496-ffc2-4742-b3f3-f971a15fd763"}
{"status": "ok", "uuid": "bc78e5bb-5ee8-4590-a9ff-b2deb41e35c8"}
{"status": "ok", "uuid": "5c273ad6-e1f3-4dd6-b2d2-a19087799ad2"}
{"status": "ok", "uuid": "ddb7a568-c06e-4ea3-8794-e1dc45e98d8b"}
{"status": "ok", "uuid": "1b1fd1beb-dc47-4f91-ba7f-c30c544bela5"}
{"status": "ok", "uuid": "07f2a2c5-448b-45ae-a9d2-0a6bfff1e08"}
{"status": "ok", "uuid": "85ab5bef-5823-4b3f-8926-a48b933368cf"}
{"status": "ok", "uuid": "df281598-dc0b-448a-aa6f-22df27c96b85"}
• (.venv APZ general) julfy@julyf-Nitro-AN515-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$
```

Logging-service instance #1. Shows successful self-registration with config-server ("Успішно зареєстровано <http://ls1:8000> в config-server"), then reception of a subset of messages (msg2, msg5, msg8, msg9). 200 OK responses on POST /log confirm Hazelcast Map writes

```
• (.venv APZ general) julfy@julyf-Nitro-AN515-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$ docker logs ls1
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [1]
Успішно зареєстровано http://ls1:8000 в config-server
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Успішно зареєстровано http://ls1:8000 в config-server
Отримано повідомлення: msg2 з UUID: 33056a56-28f4-408f-a566-e0011b94feda
INFO: 172.18.0.9:34026 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg5 з UUID: 5c273ad6-e1f3-4dd6-b2d2-a19087799ad2
INFO: 172.18.0.9:34028 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg8 з UUID: 07f2a2c5-448b-45ae-a9d2-0a6bfff1e08
INFO: 172.18.0.9:34032 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg9 з UUID: 85ab5bef-5823-4b3f-8926-a48b933368cf
INFO: 172.18.0.9:34036 - "POST /log HTTP/1.1" 200 OK
```

Logging-service instance #2 received a different subset (msg1, msg3, msg4, msg5, msg6, msg7, msg8, msg10) plus GET /log requests. This proves facade-service is round-robin / random-selecting targets from the dynamic list returned by config-server

```
• (.venv APZ_general) julfy@julfy-Nitro-AN515-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$ docker logs ls2
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Успішно зареєстровано http://ls2:8000 в config-server
Отримано повідомлення: msg1 з UUID: 92dd4063-64dd-43e4-8c1a-36e5c0276beb
INFO: 172.18.0.7:59546 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg4 з UUID: 8ead1883-3a3b-4a78-a602-3b43ecbf201f
INFO: 172.18.0.7:59550 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg5 з UUID: a15fdb80-3788-4dcd-869f-583148c857a8
INFO: 172.18.0.7:59566 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg6 з UUID: 97206662-0498-4857-8ca3-9dd01272f4ce
INFO: 172.18.0.7:59578 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg7 з UUID: 7e69b35b-ae8c-4098-ba5c-0f72b074eb95
INFO: 172.18.0.7:59580 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg10 з UUID: 00f88d4b-235e-4867-a00a-772019f5fc38
INFO: 172.18.0.7:59596 - "POST /log HTTP/1.1" 200 OK
INFO: 172.18.0.7:35324 - "GET /log HTTP/1.1" 200 OK
INFO: 172.18.0.7:45102 - "GET /log HTTP/1.1" 200 OK
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [1]
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Успішно зареєстровано http://ls2:8000 в config-server
Отримано повідомлення: msg1 з UUID: e6f5b6ef-452e-44e3-bdc4-34bd1e8a6f04
INFO: 172.18.0.9:48114 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg3 з UUID: 62aff496-ffc2-4742-b3f3-f971a15fd763
INFO: 172.18.0.9:48116 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg4 з UUID: bc78e5bb-5ee8-4590-a9ff-b2deb41e35c8
INFO: 172.18.0.9:48122 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg6 з UUID: ddb7a568-c06e-4ea3-8794-e1dc45e98d8b
INFO: 172.18.0.9:48134 - "POST /log HTTP/1.1" 200 OK
Отримано повідомлення: msg10 з UUID: df281598-dc0b-448a-aa6f-22df27c96b85
INFO: 172.18.0.9:48146 - "POST /log HTTP/1.1" 200 OK
• (.venv APZ_general) julfy@julfy-Nitro-AN515-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$ docker logs ls2
```

Logging-service instance #3 received msg7. Together with ls1 and ls2 logs, this demonstrates the requirement "messages are received by different logging-service instances" messages are replicated via Hazelcast Map but each POST lands on exactly one instance chosen dynamically

```

(.venv_APZ_general) julfy@julyfy-Nitro-ANS15-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$ docker logs ls3
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [1]
Успішно зареєстровано http://ls3:8000 в config-server
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Успішно зареєстровано http://ls3:8000 в config-server
Отримано повідомлення: msg7 з UUID: 1bfd1beb-dc47-4f91-ba7f-c30c544bela5
INFO: 172.18.0.9:52018 - "POST /log HTTP/1.1" 200 OK
(.venv_APZ_general) julfy@julyfy-Nitro-ANS15-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$

```

GET returns the aggregated message list from any logging-service (read via Hazelcast Map values) plus "Пахунок: Total messages in DB: 10" from counter-service. Confirms both read paths work and the PostgreSQL row count matches the 10 POSTs

```

(.venv_APZ_general) julfy@julyfy-Nitro-ANS15-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$ curl -X GET http://localhost:8000/facade
msg6, msg2, msg10, msg5, msg9, msg8, msg3, msg1, msg7, msg4 | Пахунок: Total messages in DB: 10(.venv_APZ_general) julfy@julyfy-Nitro-ANS15-57:~/Documents/6th_term/APZ/
Task_4-Microservices_with_MessageQueue/APZ_labs$

```

Counter-service is frozen via `docker pause`. A new POST (msg11_queued) still returns `{"status":"ok"}` instantly — proving fault tolerance: the facade only drops the payload into Hazelcast Queue and does not depend on counter-service availability

```

Task_4-Microservices_with_MessageQueue/APZ_labs$ docker pause cs
cs
(.venv_APZ_general) julfy@julyfy-Nitro-ANS15-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$ curl -X POST -H "Content-Type: application/json" -d '{"msg\":"msg11_queued\"}" http://localhost:8000/facade
{"status":"ok","uuid":"e9464ee3-4944-472d-8e1f-ff355cfb45ad"}(.venv_APZ_general) julfy@julyfy-Nitro-ANS15-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$

```

GET returns the message list (still served by logging-service) but "Пахунок: null" because counter-service cannot respond. Exactly matches the spec: "GET-запити ... мають повертати null у якості значень на рахунок"

```

(.venv_APZ_general) julfy@julyfy-Nitro-ANS15-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$ curl -X GET http://localhost:8000/facade
msg6, msg2, msg10, msg5, msg9, msg8, msg3, msg1, msg7, msg11_queued, msg4 | Пахунок: null(.venv_APZ_general) julfy@julyfy-Nitro-ANS15-57:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$

```

After unpause, counter-service consumer thread drains the Hazelcast Queue ("Збережено з черги: msgN" lines). Demonstrates automatic catch-up: buffered messages are persisted to PostgreSQL without any manual retrigger

```
• -Microservices_with_MessageQueue/APZ_labs$ docker unpause cs
cs
• (.venv_APZ_general) julfy@julfy-Nitro-AN515-S7:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$ docker logs cs
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [1]
Таблицю messages успішно ініціалізовано.
Запуск слухача черги Hazelcast...
Успішно зареєстровано в config-server
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Таблицю messages успішно ініціалізовано.
Запуск слухача черги Hazelcast...
Успішно зареєстровано в config-server
Збережено з черги: msg1
Збережено з черги: msg2
Збережено з черги: msg3
Збережено з черги: msg4
Збережено з черги: msg5
Збережено з черги: msg6
Збережено з черги: msg7
Збережено з черги: msg8
Збережено з черги: msg9
Збережено з черги: msg10
INFO: 172.18.0.9:54108 - "GET /message HTTP/1.1" 200 OK
INFO: 172.18.0.9:33520 - "GET /message HTTP/1.1" 200 OK
• (.venv_APZ_general) julfy@julfy-Nitro-AN515-S7:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$
```

"Total messages in DB: 11" — msg11_queued has been processed from the queue after recovery. Closes the fault-tolerance loop: zero messages lost during the outage

```
• (.venv_APZ_general) julfy@julfy-Nitro-AN515-S7:~/Documents/6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$ curl -X GET http://localhost:8000/facade
msg6, msg2, msg10, msg5, msg9, msg8, msg3, msg1, msg7, msg11 queued, msg4 | Рахунок: Total messages in DB: 11(.venv_APZ_general) julfy@julfy-Nitro-AN515-S7:~/Documents
• /6th_term/APZ/Task_4-Microservices_with_MessageQueue/APZ_labs$
```

https://github.com/Bohdanok/APZ_labs/tree/micro_mq